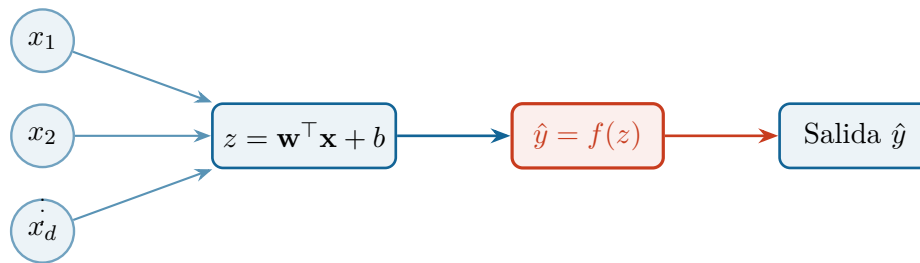


---

## Tarea 2

### La Neurona No Lineal: Funciones de Activación y Predicción de Comportamientos Desconocidos

---



*“Una neurona lineal es un microscopio que solo ve líneas rectas.  
Una función de activación le da ojos para ver curvas, umbrales y  
patrones  
que ninguna ecuación conocida podría describir de antemano.”*

## 1. Punto de Partida: ¿Qué Cambió?

### Conexión con la Tarea Anterior

En la Tarea 1 construiste una neurona que calcula  $\hat{y} = wx + b$  y la entrenaste para predecir datos **lineales y cuadráticos**. Para el caso cuadrático, tuviste que aplicar la *linealización de características* ( $x \rightarrow [x, x^2]^\top$ ) porque la neurona por sí sola no podía capturar la curvatura.

En esta tarea harás **una sola modificación** al modelo: añadirás una **función de activación**  $f$  a la salida de la neurona. Verás que este cambio mínimo desbloquea una capacidad radicalmente mayor.

El nuevo modelo es:

$$\hat{y} = f(z), \quad z = \mathbf{w}^\top \mathbf{x} + b \quad (1)$$

donde  $f: \mathbb{R} \rightarrow \mathbb{R}$  es la función de activación que tú eliges. La ecuación lineal  $z = \mathbf{w}^\top \mathbf{x} + b$  sigue siendo la misma de la tarea anterior; lo único nuevo es que **su resultado pasa por  $f$  antes de convertirse en la predicción**.

### La idea central de esta tarea

Cuando los datos provienen de un fenómeno cuya ecuación matemática **no se conoce** —o es demasiado compleja para modelarla analíticamente— una neurona con la función de activación adecuada puede **aproximar ese comportamiento de forma empírica**, usando solo los datos observados. No necesitas saber la ecuación: dejas que el entrenamiento encuentre  $w$  y  $b$  tales que  $f(wx + b)$  se parezca lo más posible a los datos.

## 2. Fundamento Matemático

### 2.1. Funciones de activación comunes

La Tabla 1 presenta las funciones que deberás implementar **desde cero** (solo NumPy).

### 2.2. Propagación hacia adelante

Con  $N$  ejemplos en forma matricial ( $\mathbf{X} \in \mathbb{R}^{N \times d}$ ,  $\mathbf{y} \in \mathbb{R}^N$ ):

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b \cdot \mathbf{1} \in \mathbb{R}^N \quad (2)$$

$$\hat{\mathbf{y}} = f(\mathbf{z}) \quad (\text{aplicada elemento a elemento}) \quad (3)$$

| Nombre               | Fórmula $f(z)$                                                      | Derivada $f'(z)$                                                  | Rango                      |
|----------------------|---------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------|
| Identidad (lineal)   | $z$                                                                 | 1                                                                 | $(-\infty, +\infty)$       |
| Sigmoide             | $\frac{1}{1 + e^{-z}}$                                              | $f(z)(1 - f(z))$                                                  | $(0, 1)$                   |
| Tangente hiperbólica | $\tanh(z)$                                                          | $1 - \tanh^2(z)$                                                  | $(-1, 1)$                  |
| ReLU                 | $\max(0, z)$                                                        | $\mathbf{1}[z > 0]$                                               | $[0, +\infty)$             |
| Leaky ReLU           | $\max(\alpha z, z), \alpha \ll 1$                                   | $\begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases}$        | $(-\infty, +\infty)$       |
| ELU                  | $\begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$ | $\begin{cases} 1 & z > 0 \\ f(z) + \alpha & z \leq 0 \end{cases}$ | $(-\alpha, +\infty)$       |
| Swish                | $z \cdot \sigma(z)$                                                 | $f(z) + \sigma(z)(1 - f(z))$                                      | $\approx (-0,28, +\infty)$ |

Cuadro 1: Funciones de activación a implementar.  $\sigma(z)$  denota la sigmoide.

### 2.3. Función de pérdida y retropropagación

Seguimos usando el MSE:

$$\mathcal{L} = \frac{1}{N} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 \quad (4)$$

Aplicando la regla de la cadena, los gradientes ahora incluyen la derivada de  $f$ :

$$\boldsymbol{\delta} = \frac{2}{N} (\hat{\mathbf{y}} - \mathbf{y}) \odot f'(\mathbf{z}) \quad (5)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{X}^\top \boldsymbol{\delta} \quad (6)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N \delta_i \quad (7)$$

donde  $\odot$  denota el producto de Hadamard (elemento a elemento). Nota que si  $f$  es la identidad,  $f'(z) = 1$  y recuperas exactamente los gradientes de la Tarea 1.

### 2.4. Escala de la salida y elección de la función de pérdida

Algunas funciones de activación restringen el rango de  $\hat{y}$ . Si los datos objetivo  $y$  no están en ese rango, el entrenamiento fallará sin importar cuántas épocas uses. Por ejemplo:

- **Sigmoide:**  $\hat{y} \in (0, 1)$ . Los datos  $y$  deben estar normalizados en ese intervalo, o debes usar la sigmoide solo como función intermedia.
- **ReLU:**  $\hat{y} \geq 0$ . Adecuada para datos estrictamente positivos.

- **Tanh:**  $\hat{y} \in (-1, 1)$ . Normaliza  $y$  al intervalo  $[-1, 1]$ .
- **Identidad, Swish, ELU, Leaky ReLU:** sin restricción fuerte de rango.

#### Para reflexionar

¿Qué función de activación elegirías si los datos tienen un comportamiento de *saturación* (crece rápido al inicio y luego se aplana)? ¿Y si tienen un umbral claro donde pasan de “cero” a “activo”? Esta elección es parte del diseño del modelo.

## 3. Descripción de la Tarea

### 3.1. Parte 1 — Implementación Extendida de la Neurona

Toma tu clase `Neuron` de la Tarea 1 y extiéndela para soportar funciones de activación intercambiables.

#### Requisitos:

- Implementa **todas** las funciones de la Tabla 1 como clases separadas que heredan de `ActivationFunction`.
- Asegúrate de que con `activation=Identity()` tu neurona produce resultados idénticos a la Tarea 1.
- Realiza un **gradient check numérico** para al menos dos funciones de activación distintas (sigmoide y ReLU, por ejemplo).

### 3.2. Parte 2 — Generación de Datos con Comportamiento Desconocido

La motivación central de usar funciones de activación es atacar problemas donde **la forma matemática de los datos no se conoce de antemano**. Para simular esto, genera al menos **tres** conjuntos de datos con las siguientes formas (sumando ruido gaussiano  $\varepsilon \sim \mathcal{N}(0, \sigma^2)$  en todos):

#### D1. Sigmoidal / de saturación:

$$y = \frac{A}{1 + e^{-k(x-x_0)}} + \varepsilon$$

Representa fenómenos que crecen rápido y luego se estabilizan (p. ej. adopción de tecnología, crecimiento poblacional con límite).

#### D2. Con umbral / escalón suavizado:

$$y = A \cdot \tanh(B(x - x_0)) + \varepsilon$$

Representa fenómenos con un cambio brusco de régimen.

**D3. Libre elección:** diseña un conjunto de datos propio cuya “ecuación verdadera” **no reveles** hasta el análisis final. Puede ser una combinación de funciones, una serie temporal simulada, o datos con estructura de umbral asimétrico. El reto es ver si tu neurona lo aprende sin conocer la fórmula.

Para cada conjunto:

- Genera  $N \geq 300$  puntos con  $x \in [-6, 6]$ .
- Divide en 80 % entrenamiento / 20 % prueba.
- Grafica los datos con la curva verdadera (sin ruido) superpuesta.
- Experimenta con dos niveles de ruido y comenta el efecto.

### 3.3. Parte 3 — Entrenamiento y Comparación de Activaciones

Para cada conjunto de datos (D1, D2, D3), entrena tu neurona con **cada una** de las funciones de activación de la Tabla 1 y compara los resultados.

**Para cada experimento, genera:**

- G1. Curva de pérdida:** MSE vs. épocas para todas las activaciones en la misma gráfica (una línea de color distinto por activación). Usa escala logarítmica en el eje  $y$ .
- G2. Ajuste del modelo:** dispersión de los datos de entrenamiento, datos de prueba y curva predicha por cada activación. Muestra en el mismo panel o en subplots comparativos.
- G3. Tabla resumen:** MSE final en prueba, épocas hasta convergencia y si el modelo divergió o no, para cada activación.

**Pregunta guía:** ¿Hay una función de activación que gane siempre? ¿O depende de la forma de los datos?

### 3.4. Parte 4 — Extensión a Múltiples Entradas

Hasta ahora has trabajado con una sola variable de entrada ( $d = 1$ ). En esta parte ampliarás a  $d > 1$  para mostrar que la misma neurona escala.

Genera datos 2D de la forma:

$$y = f_{\text{true}}(x_1, x_2) + \varepsilon \quad (8)$$

donde  $f_{\text{true}}$  puede ser, por ejemplo:

$$y = \sigma(2x_1 - 3x_2 + 1) + \varepsilon \quad \text{o} \quad y = \tanh(x_1^2 - x_2) + \varepsilon$$

(cualquier función no lineal de dos variables).

- Entrena tu neurona con  $\mathbf{x} = [x_1, x_2]^\top$  y la activación que mejor resultó en la Parte 3.
- Grafica la superficie de predicción  $\hat{y}(x_1, x_2)$  frente a la superficie verdadera  $y_{\text{true}}(x_1, x_2)$  usando `ax.plot_surface()` o un mapa de calor.
- Experimenta con  $d = 3$  y comenta: ¿qué cambia en el entrenamiento?

### 3.5. Parte 5 — Notebook Interactivo con Slides

Esta es la parte central de presentación. El notebook debe funcionar como una **demonstración interactiva** que cualquier persona pueda ejecutar y explorar.

| Widget                       | Tipo     | Qué debe actualizarse en tiempo real               |
|------------------------------|----------|----------------------------------------------------|
| Función de activación        | Dropdown | Curva $f(z)$ , ajuste del modelo, curva de pérdida |
| Tasa de aprendizaje $\alpha$ | Slider   | Velocidad y estabilidad de convergencia            |
| Número de épocas             | Slider   | Punto de parada; subajuste vs. convergido          |
| Nivel de ruido $\sigma$      | Slider   | Dispersión de los datos y MSE final                |
| Conjunto de datos            | Dropdown | Cambia entre D1, D2, D3                            |
| Número de entradas $d$       | Selector | Muestra cómo escala la neurona a más variables     |
| Inicialización de $w_0$      | Slider   | Trayectoria en la superficie de pérdida            |

Cuadro 2: Widgets mínimos requeridos en el notebook interactivo.

#### Widgets obligatorios —

**Panel de visualización en tiempo real** — Cada vez que el usuario mueva un slider o cambie un dropdown, el panel debe actualizar **simultáneamente**:

- La gráfica de la función de activación seleccionada (con su derivada superpuesta).
- El ajuste del modelo sobre los datos.
- La curva de pérdida hasta la época seleccionada.
- Los valores actuales de  $w$  y  $b$  aprendidos.

**Estructura de las slides** — Organiza el notebook con celdas marcadas como diapositivas (RISE o nbconvert). La estructura mínima es:

1. **Portada:** título, nombres, motivación en una frase.
2. **El problema:** ¿qué pasa cuando no conoces la función? Demo visual.
3. **La solución:** neurona + función de activación. Diagrama animado.
4. **Implementación:** muestra el código clave con comentarios.
5. **Demo interactiva D1** (sigmoideal): explorer de hiperparámetros.
6. **Demo interactiva D2** (umbral): comparación de activaciones.
7. **Demo interactiva D3** (libre): el reto — ¿puede la neurona aprenderlo?
8. **Múltiples entradas:** superficie 3D interactiva.
9. **Conclusiones:** cuándo usar cada activación, limitaciones de una neurona.

### 3.6. Parte 6 — Análisis y Reflexión (Bonus: 10 pts adicionales)

Responde en el notebook (en celdas Markdown dentro de las slides):

- R1.** Compara la curva de pérdida de la neurona lineal (Tarea 1) con la de la neurona activada sobre los mismos datos cuadráticos. ¿La función de activación sustituye a la linealización de características? ¿O la complementa?
- R2.** Encuentra experimentalmente un valor de  $\alpha$  para el cual ReLU diverge pero Sigmoid converge en el mismo conjunto de datos. ¿A qué se debe esta diferencia?
- R3.** Para el conjunto D3 (libre), revela la ecuación verdadera al final de la presentación. ¿Cuánto se alejó el modelo? ¿Qué función de activación se acercó más?
- R4.** ¿Qué le falta a *una sola* neurona activada para modelar fenómenos verdaderamente complejos (como reconocimiento de imágenes o lenguaje natural)? Conecta tu respuesta con la motivación de las redes neuronales profundas.

### Recomendaciones técnicas

- **Reproducibilidad:** fija `np.random.seed(42)` al inicio.
- **Widgets:** usa `ipywidgets.interact` o `ipywidgets.interactive_output` con Output para controlar qué celdas se actualizan sin rerenderizar todo el notebook.
- **Gráficas:** para la superficie 3D usa `matplotlib` con `Axes3D` o `plotly.graph_objects.Surface` (permite rotar interactivamente).
- **Overflow numérico:** en la sigmoide, recorta  $z$  con `np.clip(z, -500, 500)` para evitar NaN o inf.

- **Convergencia de ReLU**: inicializa  $w$  con valores pequeños ( $\sim 0,01$ ) para evitar neuronas muertas desde el inicio.
- **Slides**: marca cada celda como **Slide**, **Sub-slide** o **Fragment** en la vista de *Slideshow* de Jupyter.

## 4. Conexión con el Panorama General

---

Al terminar esta tarea habrás construido, desde cero y con plena comprensión matemática, el bloque fundamental de **cualquier red neuronal profunda**. Una red neuronal no es más que muchas de estas neuronas activadas, apiladas en capas:

$$\mathbf{a}^{(1)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{a}^{(2)} = f(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}), \quad \dots$$

Cada capa transforma la representación de los datos de forma no lineal, permitiendo que la red aprenda jerarquías de características cada vez más abstractas. La función de activación es la pieza que hace posible todo esto.

Lo que has hecho en estas dos tareas —una neurona lineal y luego una neurona activada— es exactamente el camino que siguieron los investigadores que desarrollaron el *backpropagation* en los años 80. A partir de aquí, el salto hacia redes profundas es conceptualmente pequeño, aunque computacionalmente poderoso.

---

*La mejor forma de entender una herramienta es construirla.*

*La mejor forma de convencer a otros es mostrársela en vivo.*

*Eso es exactamente lo que esta tarea te pide.*

---